National Technical University of Athens

School of Electrical and Computer Engineering

Semester 7th

Final Project for the Multimedia Technology Course

Application Description

As part of this project, a Task Management System will be developed. The application will allow users to create, edit, and monitor available tasks. In addition, users will be able to manage multiple tasks, set priorities and deadlines, and receive reminders for upcoming tasks.

A.1. Logic Design and Implementation (40%)

The following describes the features the application should provide to the user regarding the creation and management of tasks and reminders.

<u>Adding, editing, and deleting tasks</u>

The user will be able to create new tasks. The relevant information for a task should include: title, description, category, priority, completion deadline, and status. For simplicity, the completion deadline for a task will not include a time, but will be defined at the day level (e.g., 12/14/24).

The status of a task will always be one of the following: *"Open", "In Progress"*, *"Postponed"*, *"Completed"*, and *"Delayed"*. For each new task, the default status will be *"Open"*. Upon application startup, tasks that are not *"Completed"* and have passed their completion deadline should be identified, and their status should automatically be changed to *"Delayed"*.

The user will be able to edit all the details of a task, as well as to delete any tasks. When deleting tasks, be sure to update any reminders that have been set for the task to be deleted.

Adding, editing, and deleting categories

The user will be able to create new categories by entering a name for them. In addition, the user will be able to edit a category's name. For simplicity's sake, we assume that there are no subcategories.

In addition, it will be possible to delete a category along with the automatic deletion of all tasks belonging to it. In this case, the reminders for the deleted tasks must be updated accordingly.

Adding, editing, and deleting priorities

The application should include a predefined priority level named *"Default"*. The user will be able to create new priority levels by assigning them a name. In addition, the user will be able to rename and delete priority levels.

The rename and delete functions will apply to all priority levels except the default one. Moreover, when a priority level is deleted, all tasks belonging to that level will automatically be assigned the default priority level.

Additional features

Setting and managing reminders: Users will be able to create reminders for tasks. A reminder will always be associated with a specific task, and multiple reminders can be set for a single task. If a task has a *"Completed"* status, it will not be possible to set reminders. When the user changes a task's status to *"Completed"*, the application will automatically delete any reminders associated with that specific task.

The application should support the following types of reminders: (i) one day before the deadline, (ii) one week before the deadline, (iii) one month before the deadline, (iv) a specific date set by the user. Appropriate checks must be implemented to ensure that the selected reminder type makes sense based on the task completion deadline, and if there is an issue, the user must be notified with the corresponding message. Finally, the user will be able to edit and delete reminders.

Task search: Users will be able to search for tasks based on any combination of the following criteria: title, category, and priority.

A.2. Storing and Retrieving Application Information (10%)

A file-based solution that stores data in JSON format will be used to store and retrieve the application's information.

JSON (JavaScript Object Notation) is a data representation format widely used for storing and transferring data. It is easy for humans to read and, at the same time, understandable by several programming languages. JSON follows a simple structure, using text to represent data in a key-value format. You can find more information in the workshop slides under the *"Java I/O"* section.

First, you need to decide on and define your own data schema for the JSON data, as well as the set of files that will be used to store the application's data. The files containing the application's data should be stored in a folder named ***"medialab"***.

Next, you must implement, through the appropriate classes, the methods that will allow you to retrieve the information contained in the files, initialize the relevant objects in your application, and refresh the files so that the overall state can be maintained across consecutive executions of the application.

Below, we describe the logic that must be implemented to retrieve and refresh the application's data.

- <u>Application initialization</u>: You must retrieve all the information contained in the JSON files and, at the same time, initialize the corresponding objects in your application.
- <u>Application execution</u>: The application will use the status information retrieved into program memory during initialization. All functions related to the tasks and reminders managed by the application will be executed based on the information stored in program memory.
- <u>Application termination</u>: JSON files containing system status information will be updated exclusively before the application is terminated. The implementation must save the application's overall status at the time of termination to the corresponding JSON files.

A.3. Graphical User Interface Development (30%)

You must design and implement an appropriate graphical user interface (GUI) using the JavaFX framework [1][2].

Note: The following outlines the basic specifications for the graphical user interface. For all details regarding the final implementation, you may make any choices you wish regarding the appearance and overall user interaction with the application, without any impact on the final grade. For example, you can choose a simple display for the various elements or combine various features from JavaFX to create a result that corresponds to a modern application. In any case, there is no need to make this part of the assignment overly complex.

Initially, when the application starts up, if there are tasks in the “Delayed” status, the user should be notified via a pop-up window indicating the number of overdue tasks.

When creating the graphical user interface, please follow these general guidelines:

- Create the main "window" of the application titled "MediaLab Assistant" and set the appropriate dimensions.
- Divide the window into two main sections.
- The top part of the screen will display summary information that should be updated accordingly based on the user’s actions. The information includes: (i) total number of tasks regardless of status, (ii) number of tasks with “Completed” status, (iii) the number of tasks with a “Delayed” status, and (iv) the number of tasks with a completion deadline within 7 days.
- The other part of the screen should implement the various functions that the graphical interface must support. You have complete freedom regarding how to implement these specific functions; however, you must ensure that the graphical interface is updated appropriately based on the user’s actions.

The graphical user interface should support the following functions:

- Task management: the application must display available tasks by category. It should also allow the user to create a new task, modify a task, and delete a task. The implementation must follow the logic described in Section A.1.
- Category management: the application must display the list of categories. Furthermore, the user should be able to define new categories, rename a category, and delete categories. The implementation must follow the logic described in Section A.1.
- Priority level management: The application must display the available priority levels. The user should be able to modify the names of priority levels and delete

priority levels. The implementation must follow the corresponding logic from Section A.1.

- Reminder management: The application must display all active reminders. The user should be able to edit a reminder and delete reminders. The implementation must follow the logic described in Section A.1.
- Task search: There must be a form that allows users to search for tasks based on the specifications in Section A.1. The results must include: title, priority level, category, and completion deadline.

Finally, the implementation of your graphical interface upon application termination must ensure that system information is updated in the relevant files, in accordance with the procedure described in Section A.2.

A.4. Other Requirements (20%)

- The implementation must adhere to the principles of object-oriented programming (OOP design principles).
- In a class of your choice, every public method it contains must be documented according to the specifications of the javadoc tool [3].

Note: For anything that is not clear from the statement, you may make your own assumptions and inferences. The statement describes the basic requirements for the application; however, you may make your own design assumptions in an effort to make the application more realistic, without making the implementation overly complex.

Deliverables

- The project (in the IDE of your choice) containing the application's source code.
- A brief (maximum 3 pages) report containing a general description of the implementation design and a description of your design regarding the structure of the information in the various JSON files. You should also mention any functionality you have not implemented, as well as any assumptions you have made. Under no circumstances should you include code snippets.

References

[1] https://docs.oracle.com/javase/8/javafx/get-started-tutorial/jfx-overview.htm

[2] https://docs.oracle.com/javafx/2/get_started/jfxpub-get_started.htm

[3] https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html